BirthdayCollator is a lightweight full-stack application built with **.NET 9**, **C#**, and a **React (Vite)** frontend. It retrieves and normalizes significant (round number or aged) birthday data from date-based sources, applies consistent cleanup rules, and presents results through a modern UI. Users can optionally generate AI-powered summaries for each person by providing their own OpenAI API key. The project emphasizes clean architecture, async workflows, reusable components, and predictable data pipelines.

Features

- **.NET 9 REST API** for retrieving birthdays by month/day
- **Year override** support for testing and historical lookups
- **IncludeAll mode** for expanded parsing and edge-case handling
- **Async fetch pipeline** with cancellation support
- **In-memory caching** to avoid repeated parsing
- **Text-cleanup utility** for consistent, readable output
- **Modern React UI** with reusable components and hooks
- **Optional OpenAI summaries** using user-provided API keys
- **AI endpoints exposed via Swagger** for testing and development

Tech Stack

Backend

- .NET 9
- C#
- In-memory caching
- Async/await pipelines
- Clean controller + provider structure
- OpenAI integration (server-side summarize endpoint)

Frontend

- React (Vite)

- Custom hooks
- Reusable components
- AbortController for cancelable fetches
- CSS modules
- Client-side API key entry for OpenAI usage
-

Architecture Overview

Backend

- **BirthdaysController** Handles birthday lookups, includeAll flag, caching, and year overrides.
- **AIController**
  - GET /api/ai/has-key — checks whether an API key is configured
  - POST /api/ai/summarize — generates a summary for a person using OpenAI
- **BirthSourceEngine / YearBirthSource** Loads and parses birthday data for specific years, including leap-year logic.
- **Parser + Cleanup Pipeline** Normalizes raw text, removes noise, and produces consistent person objects.

Frontend

- **Toolbar** Month/day selectors, includeAll toggle, and year override input.
- **ResultsList / ResultItem** Displays people, milestone highlighting, and summary controls.
- **Summaries Hook** Manages OpenAI summary generation and local state.

This structure keeps the UI simple while the backend handles parsing, filtering, and business logic.

AI Summary Feature

Users can optionally enter their **OpenAI API key** in the UI. When provided:

- The app enables **Summarize** buttons for each person.

- Clicking **Summarize** sends the person's data to the backend's /api/ai/summarize endpoint.
- The backend calls OpenAI and returns a short, readable summary.
- Summaries are cached client-side until cleared.
- No API keys are stored on the server.

The app works fully without an API key; summaries are optional.

Running the Project

Backend

bash

cd BirthdayCollator.Server

dotnet restore

dotnet run

The API will start on https://localhost:5001 (or the port in launchSettings.json). Swagger UI is available at /swagger.

Frontend

bash

cd birthdaycollator-ui

npm install

npm run dev

The UI will start on http://localhost:5173.

API Endpoints

Birthdays

- GET /api/Birthdays
- GET /api/Birthdays/exists/{month}/{day}

- GET /api/Birthdays/years
- GET /api/Birthdays/current
- GET /api/Birthdays/override
- POST /api/Birthdays/override
- DELETE /api/Birthdays/{month}/{day}

AI

- GET /api/ai/has-key
- POST /api/ai/summarize

Using the App

1. Select a **month** and **day**.
2. Optionally enter a **year override**.
3. Toggle **Include All** to expand parsing.
4. View normalized results in the list.
5. (Optional) Enter an **OpenAI API key** to enable summaries.
6. Click **Summarize** on any person to generate an AI summary.

Birthdays

Today + - Run

March 3

12 results

94 Roy Fisher - Australian rugby league player

Summarize

91 Gene Bess - born in Oak Ridge, Missouri

Summarize

91 Mal Anderson - Australian tennis player

Summarize